LakeTemperature Testing Software

Slides by Aidan Bensch

Background

- We are testing a module of E3SM, an environmental model. This module, LakeTemperature, is part of the greater ELM (E3SM Land Model) which was originally a fork of the CLM (Community Land Model).
- We are able to test specifically this module using SPEL, a software that adds a NetCDF interface for the constants, inputs, and outputs of a module of E3SM and compiles that specific part of the model. The executable produced by this is named `elmtest.exe`.

Project Goals

- Develop a software that can systematically change the constants used in the LakeTemperature model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find inputs that will cause physically impossible or unrealistic outputs. Apply assertions pertaining to the values of constants, inputs, and outputs.

Our Testing Solution

- Automates the process of modifying constants in `spel-constants0001.nc` and the process of analyzing the resulting values in `fut-outputs0001.nc`.
- Both the process of choosing values for each constant and the process of analyzing the resulting outputs is extendable through sampling plugins and analysis plugins respectively.

What is a Sample?

- A sample is a set of values for the constants contained in `spel-constants0001.nc`.
- `elmtest.exe` is run once for each sample.

Constants (formerly lakeparams.txt)

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-005
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample (JSON used for illustration)

```
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

What Outputs are Received per Sample?

- Analysis plugins will receive the outputs from the `spel-outputs0001.nc` file in the form of multidimensional numpy arrays.

Outputs Received by Analysis Plugins (JSON for illustration)

```
{
  "veg_ef%eflx_soil_grnd": [0.0, ...],
  "veg_ef%eflx_gnet": [[0.0, ...], ...],
  ...
}
```

Sample Groups

- Represents an ordered grouping of samples
- Analysis plugins can analyze the result of multiple consecutive runs of elmtest.exe in context this way.

Sample Group Data (JSON for illustration)

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data (JSON for illustration)

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

Plugins

- Our testing framework supports two types of plugins, sampling and analysis. Sampling plugins define values for constants, and analysis plugins view the outputs of the model.

Sampling Plugin Interface

- Sampling plugins define a subclass of Sampler, returning one or more SampleGroups, which are sequences of individual samples.

Analysis Plugin Interface

- Analysis plugins can either be “per sample” or “sample group” analysis plugins.
- “Per sample” plugins define a subclass of PerSampleAnalyzer, which is called each time elmtest is run, and receives the outputs produced by that specific sample.
- “Sample group” plugins define a subclass of SampleGroupAnalyzer, which is called after all samples in a group have been run through elmtest, and it receives all outputs from each.